

Stress testing PROXAI provenance capture

Branch `ssg-lugia-stress-test`, PR #3, issue #2.

Summary

I built a framework to test whether PROXAI's provenance graphs can identify which stage of a preprocessing pipeline caused a change in the pipeline's output. The framework runs a pipeline repeatedly, changing one parameter per run, and compares the resulting provenance graphs against a baseline. I connected two pipelines to it: SSG-LUGIA, a genomic island predictor, and a scikit-learn preprocessing and training pipeline on the Census income dataset.

The per-run output comparison works and gives a clear reading of which pipeline parameters affect the result. The provenance-based stage attribution does not work reliably yet. The main obstacle is PROXAI's activity extraction: it depends on an LLM call that returns a different structure on each run and fails without warning when the structure is malformed. The test also exposed five robustness problems in PROXAI, filed as issue #2 and partly fixed on the branch.

Background

PROXAI records provenance while a preprocessing pipeline runs and stores it in Neo4j as a graph of activities, columns, and entities. The pipeline author subscribes a DataFrame to a tracker and calls `analyze_changes` at each stage boundary; PROXAI diffs consecutive snapshots and builds the graph, using an LLM to name and describe the operations it sees. The example pipelines shipped with PROXAI are all tabular data-cleaning scripts.

The task was to stress test PROXAI with pipelines that are less like those examples, and to check whether the captured provenance is diagnostically useful rather than merely present. "Diagnostically useful" means: when a pipeline's output changes, can the graph point to the stage responsible, without a person inspecting each stage by hand.

Method

The approach needs pipelines whose ground truth is known. Take a pipeline with independently tunable stages, define a set of configurations that each change exactly one stage, run every configuration through PROXAI, and check whether the stage whose provenance diverges most from the baseline is the stage that was actually changed.

The framework lives in `stress_test/`. Each pipeline under test is a profile module that holds its configuration list, the path to its adapter, the environment variable its configuration is passed through, the summary line to read metrics from, and a mapping from activity code to pipeline stage. Three scripts consume a profile:

- `run_stress_test.py` runs `prolit_run.py` once per configuration. It clears Neo4j in bounded batches first, records each provenance graph to a JSON file through the Neo4j driver, and retries a configuration when it hits a known transient failure.
- `compare.py` groups each graph's activities into stages by their code, scores how far each stage's provenance diverges from the baseline, ranks the stages, and checks the top stage against the configuration's known parameter change. It writes a comparison table.
- `demo.py` runs a subset of configurations end to end and prints the table plus the Cypher queries needed to view the graph.

Adding a third pipeline requires one adapter and one profile module; the procedure is documented in `stress_test/README.md`.

The two pipelines

SSG-LUGIA runs a genome string through windowing, feature extraction, anomaly detection, and post-processing, producing numpy arrays. It does not use pandas, so the adapter wraps each stage's output as columns on a single DataFrame. The eleven configurations vary the classifier variant (F, R, P), the window size and step, and the feature representation.

The Census pipeline is a scikit-learn sequence of clean and impute, categorical encoding, feature scaling, feature selection, and model training on `datasets/census.csv`. It is deterministic. Its eleven configurations change one knob per stage: imputation strategy, encoding scheme, scaler, number of selected features, and classifier.

Results

The eleven-configuration SSG-LUGIA sweep at granularity 1 produced the output deltas below. They match the pipeline's design.

configuration	parameter changed	windows	islands	change in islands	change in windows
baseline_F	none	47,991	14		
classifier_R	classifier = SSG-LUGIA-R	47,991	34	+20	0
classifier_P	classifier = SSG-LUGIA-P	47,991	10	-4	0
window_coarse_20000	w = 20000, dw = 200	23,946	5	-9	-24,045
window_fine_5000	w = 5000, dw = 50	96,081	17	+3	+48,090
window_very_fine_1000	w = 1000, dw = 10	480,804	9	-5	+432,813
step_dense_25	dw = 25	191,962	31	+17	+143,971
feature_karlin_raw	karlin_mode = raw	47,991	14	0	0
feature_karlin_original	karlin_mode = original	47,991	13	-1	0
feature_no_entropy	entropy_features = off	47,991	12	-2	0
feature_more_pca	pca dims = 5 / 5 / 5	47,991	15	+1	0

The classifier variant moves the island count from 10 to 34 with no change to the window count, which is consistent with an effect confined to the anomaly detection stage. The window and step parameters change the window count by a factor of 0.5 to 10 and move the island count in both directions, consistent with a scanning-resolution effect. The feature-representation parameters barely move either number.

Stage localisation did not work. At granularity 1 the graphs have only tens of nodes, and the divergence score is dominated by run-to-run variation in how the LLM phrases the activity list rather than by real change in the pipeline. Nine scored configurations, zero correct.

Raising the granularity does not fix this. At granularity 3 the SSG-LUGIA baseline graph has roughly 750,000 nodes and 1.5 million relationships, which is enough structure for a meaningful comparison, but the activity layer is intermittently lost. Running the same baseline configuration twice produced 0 activity nodes on one run and 17 on the next, and both runs reported success. The cause is items 1 and 4 in the defects below.

The Census pipeline runs end to end. The baseline and three other configurations were verified individually and produced clean graphs with five activity nodes, one per stage, each correctly classified. A full eleven-configuration sweep on the Groq backend was blocked by that account's daily token limit, which the development runs had already spent.

Re-running on a local Ollama model removes the limit but takes about 100 seconds per LLM call on CPU, so the full sweep is a multi-hour job that has not completed.

Defects found in PROXAI

Filed as issue #2. Ordered by how much each blocks automated stage attribution.

1. Neo4j write errors are swallowed. The query executor catches every exception, prints `[NEO4J ERROR]`, and returns `None`. Callers do not check the return value, so a run that failed to create any activity nodes still reports success. This is the main reason a bad run is hard to detect.
2. `neo4j.delete_all()` runs a single unbatched `MATCH (n) DETACH DELETE n`. On a large graph this exceeds Neo4j's transaction memory limit and fails; because of item 1 the failure is silent, and the next run builds its graph on top of the previous one. Worked around in the harness by clearing the graph in batches before each run.
3. `column_entity_vision` indexes the activity list positionally against the tracker snapshots. When the LLM returns fewer activities than there are `analyze_changes` calls, the index runs off the end and raises `IndexError`, losing the run. Guarded on the branch.
4. The LLM activity extraction is neither validated nor deterministic. The same pipeline yields between 3 and 17 activities across runs. When the model does not fence its output the parser returns `None` and the run crashes; when the output is long it is truncated mid-structure and fails to parse. The Groq free tier compounds this with an 8,000 token-per-minute and 200,000 token-per-day limit that a sweep exhausts. On the branch this is mitigated with a tolerant parser, a bounded token limit, and a local Ollama backend as the default, but the underlying non-determinism needs a schema-constrained extraction in PROXAI core.
5. A stage is invisible to PROXAI unless it runs after `tracker.subscribe()`. The first version of the SSG-LUGIA adapter computed features before subscribing, so every feature-representation configuration changed something the graph could not see. Fixed by restructuring the adapter so it subscribes a bare per-window frame first and every stage is a tracked transformation on it.

Changes made

New framework, under `stress_test/` and `pipelines/`:

- `stress_test/ssg_lugia.py` and `stress_test/census_ml.py`, the two profiles.
- `stress_test/run_stress_test.py`, `compare.py`, `demo.py`, all taking a `--profile` argument.
- `pipelines/ssg_lugia_pipeline.py` (subscribe-first version) and `pipelines/census_ml_pipeline.py`.

- `stress_test/README.md`, `ISSUE.md`, `CHANGELOG.md`, and a compat patch for the SSG-LUGIA submodule.

New LLM handling:

- `LLM/llm_extract.py`, a tolerant parser for the model's output.
- `LLM/llm_client.py`, a backend factory. The pipeline-analysis calls default to a local Ollama model, selectable with `PROXAI_LLM_BACKEND`; Groq is still available.

Core change:

- `tracking/column_entity_approach.py` gains one guard against the activity-snapshot index mismatch in defect 3. It prevents the crash; it does not fix attribution.

Environment changes, all needed only to run PROXAI on the development machine and flagged to be made config-driven before any merge: a Groq model swap because the account had no Llama models, the Graph Chat LLM moved to Ollama, more Neo4j memory, SSG-LUGIA added as a submodule, and biopython added as a dependency.

Commits on the branch: 08baa26, da0045c, ba69736, 4d47d27, a3b234b, f34ad36, c1fce75, 0b10449, fa78096.

Limitations and next steps

The framework and the output comparison are usable now. The provenance-based stage attribution is not, and it will not be until the LLM activity extraction is made deterministic and write failures stop being silent. Those are defects 1 and 4 and belong in PROXAI core, not the harness.

Remaining work, in order:

1. Fix defects 1 and 4 in core.
2. Finish the Census eleven-configuration sweep, either on local Ollama over several hours or on Groq once the daily budget resets, and add its table.
3. Add a third pipeline, for example an NLP or image preprocessing chain.
4. Re-run both sweeps at granularity 3 once activity creation is reliable and measure stage-attribution accuracy across all pipelines.

Running it

From the repository root, with the virtual environment active and Neo4j running:

```
git submodule update --init --recursive
git -C external/SSG-LUGIA apply ../../stress_test/ssg-lugia-biopython-
compat.patch
pip install -r requirements.txt
pip install biopython
cd neo4j && docker compose up -d && cd ..

set PROXAI_LLM_BACKEND=ollama
python stress_test/run_stress_test.py --profile census_ml --dataset
datasets/census.csv
python stress_test/compare.py --profile census_ml
```

The comparison table is written to stress_test/results/_comparison.md.